Започнахме лекцията с кодиране на машина на Тюринг. Целта е да съпоставим на всяка машина на Тюринг $\mathcal{M}$ дума от $\{0,1\}^\star$, която да съдържа цялата информация за машината.

Да приемем, че:

- $Q = \{q_1, q_2, \ldots, q_n\}$, където $n \geq 2$;
- $q_1 = q_{\texttt{start}}$, $q_2 = q_{\texttt{accept}}$ и $q_3 = q_{\texttt{reject}}$;
- $\Sigma = \{x_1, \ldots, x_\ell\}$;
- $\Gamma = \{x_1, x_2, \ldots, x_s\}$, където $\ell < s$ и $x_s = \sqcup$;
- $d_1 = \square$, $d_2 = \triangleleft$, $d_3 = \triangleright$.

Първо кодираме като думи над $\{0,1\}$ преходите на машината на Тюринг.

Нека $\delta(q_i, x_j) = (q_k, x_t, d_m)$ е един от тези преходи. Кодираме го със следната дума:

$$0^i 1 0^j 1 0^k 1 0^t 1 0^m.$$

Да обърнем внимание, че в тази дума няма последователни единици и тя започва и завършва с нула.

Сега ще трябва да кодираме цялата функция на преходите δ. Забележете, че броят на преходите е $r = (n-2) \cdot s = |Dom(\delta)|$.

Нека $\texttt{code}_1$, $\texttt{code}_2$, $\ldots$, $\texttt{code}_r$ е редицата от кодовете на всички преходи, подредени спрямо каноничната наредба.

Тогава кодът на $\mathcal{M}$ е следната дума над $\{0,1\}$:

$$\ulcorner \mathcal{M} \urcorner \stackrel{\text{деф}}{=} 1110^\ell 11\ \texttt{code}_1\ 11\ \texttt{code}_2\ 11\ \cdots\ 11\ \texttt{code}_r\ 111.$$

Забележете, че разлика от числата n и s, които могат да се извлекат от кода $\ulcorner \mathcal{M} \urcorner$, информацията за ℓ трябва изрично да се добави.

Ще означаваме с $\mathcal{M}_\omega$ машината на Тюринг, чийто код е ω, т.е. искаме $\ulcorner \mathcal{M}_\omega \urcorner = \omega$.

От всяка машина $\mathcal{M}$ може да се получи ефективно нейният код $\ulcorner \mathcal{M} \urcorner$. Обратно, от дадена дума $\omega \in \{0,1\}^\star$ може да се получи ефективно машината $\mathcal{M}_\omega$ с код ω. Разбира се, не всяка дума ω е код на машина на Тюринг, но езикът

$$L_{\texttt{code}} \stackrel{\text{деф}}{=} \{\ \omega \in \{0,1\}^\star \mid \omega \text{ е код на машина на Тюринг }\}$$

е разрешим. С други думи, съществува алгоритъм, който при вход произволна дума $\omega \in \{0,1\}^\star$ определя дали ω представлява код на някоя машина на Тюринг $\mathcal{M}$.

За да можем да работим с една на машина на Тюринг на практика няма разлика дали е дадена самата машина или е даден нейният код. От друга страна, кодирането дава възможност да използваме машини на Тюринг като данни в рамките на изчислението на друга машина на Тюринг (в частност да извършваме симулация на кодираните машини).

Нека $L_{code} = \{\omega_1,\ \omega_2,\ \ldots,\ \omega_n,\ \ldots\}$. Стана ясно, че списъкът

$$\mathcal{M}_{\omega_1},\ \mathcal{M}_{\omega_2},\ \ldots,\ \mathcal{M}_{\omega_n},\ \ldots$$

съдържа всички машини на Тюринг, така че редицата

$$\mathcal{L}(\mathcal{M}_{\omega_1}),\ \mathcal{L}(\mathcal{M}_{\omega_2}),\ \ldots,\ \mathcal{L}(\mathcal{M}_{\omega_n}),\ \ldots$$

съдържа всички полуразрешими езици. Да разгледаме следната безкрайна таблица:

	$\mathcal{M}_{\omega_1}$	$\mathcal{M}_{\omega_2}$	$\ldots$	$\mathcal{M}_{\omega_n}$	$\ldots$
ω_1	$\boxed{0}$	1	$\ldots$	0	$\ldots$
ω_2	1	$\boxed{1}$	$\ldots$	0	$\ldots$
$\vdots$	$\vdots$	$\vdots$	$\ldots$	$\vdots$	$\ldots$
ω_n	0	1	$\ldots$	$\boxed{0}$	$\ldots$
$\vdots$	$\vdots$	$\vdots$	$\ldots$	$\vdots$	$\ldots$

На ред i и стълб j сме поставили 1, ако $\omega_i \in \mathcal{L}(\mathcal{M}_{\omega_j})$ и 0, ако $\omega_i \notin \mathcal{L}(\mathcal{M}_{\omega_j})$.

Тъй като всеки полуразрешим език се разпознава от някоя машина на Тюринг, по стълбовете на таблицата получаваме всички полуразрешими езици, които се състоят от думи от L_{code}. Нашата цел е да конструираме език L_{diag}, който не е полуразрешим, затова ще се погрижим той да се различава в поне една позиция от всеки стълб в таблицата.

И така на позиция $(1,1)$ имаме 0, затова ще вземем $\omega_1 \in L_{diag}$. Този избор гарантира, че $L_{diag} \neq \mathcal{L}(\mathcal{M}_{\omega_1})$, тъй като $\omega_1 \notin \mathcal{L}(\mathcal{M}_{\omega_1})$.

На позиция $(2,2)$ имаме 1, затова ще вземем $\omega_2 \notin L_{diag}$ и така гарантираме $L_{diag} \neq \mathcal{L}(\mathcal{M}_{\omega_2})$.

По същия начин постъпваме с всички останали диагонални позиции: на позиция (n,n) имаме 0 и ще вземем $\omega_n \in L_{diag}$ и т.н.

Получаваме езикът L_{diag}, който е различен от всички езици $\mathcal{L}(\mathcal{M}_{\omega_n})$, така че той не е полуразрешим.

През първия семестър видяхме аналогично разсъждение (диагонален метод на Кантор), с което показахме, че множествата от естествени числа не могат да се подредят в безкрайна редица. По-точно, за всяка редица от множества от естествени числа с диагоналния метод построихме ново множество от естествени числа, което не е член на редицата.

Теорема 1 *Диагоналният език*

$$L_{diag} \ = \ \{\ \omega \in \{0,1\}^\star \mid \omega \in L_{code}\ \&\ \omega \notin \mathcal{L}(\mathcal{M}_\omega)\ \}$$

не е полуразрешим, т.е. не се разпознава от машина на Тюринг.

Доказателство. Да допуснем, че $L_{diag} = \mathcal{L}(\mathcal{M})$. Нека $\omega = \ulcorner \mathcal{M} \urcorner$ е кодът на $\mathcal{M}$. Тогава имаме $\omega \in L_{diag} \iff \omega \notin \mathcal{L}(\mathcal{M}_\omega) \iff \omega \notin \mathcal{L}(\mathcal{M})$. Достигнахме до противоречие с $L_{diag} = \mathcal{L}(\mathcal{M})$ (по-точно, не може едно съждение да е логически еквивалентно на своето отрицание).

Теорема 2 *Езикът*

$$L_{accept} \;=\; \{\, \omega \in \{0,1\}^\star \mid \omega \in L_{code} \;\;\&\;\; \omega \in \mathcal{L}(\mathcal{M}_\omega) \,\}$$

е полуразрешим, но не е разрешим.

Доказателство. Не е възможно L_{accept} да е разрешим, тъй като $L_{diag} = L_{code} \setminus L_{accept}$ и разрешимите езици са затворени относно всички булеви операции.

За да покажем, че L_{accept} е полуразрешим, дефинираме машина на Тюринг $\mathcal{M}'$, която при вход произволна дума ω, работи по следния начин:

(1) $\mathcal{M}'$ проверява дали $\omega \in L_{code}$, т.е. дали ω е код на машина на Тюринг.

(2) Ако ω е код на машината на Тюринг $\mathcal{M}_\omega$, то $\mathcal{M}'$ симулира работата на $\mathcal{M}_\omega$ върху ω.

(3) Ако след краен брой стъпки симулацията завърши с резултат, че $\mathcal{M}_\omega$ приема думата ω, то $\mathcal{M}'$ също завършва като приема ω.

Получаваме, че за всяка дума ω е изпълнена еквивалентността

$$\omega \in L_{accept} \;\Leftrightarrow\; \omega \in \mathcal{L}(\mathcal{M}'),$$

откъдето следва, че L_{accept} е полуразрешим език. Повече подробности как се извършва симулация на машина по даден неин код ще дадем след малко.

Като следствие получаваме, че L_{accept} е полуразрешим език, чието допълнение $\overline{L_{accept}}$ не е полуразрешим език (от теоремата на Пост).

Теорема 3 (неразрешимост на стоп-проблема) *Езикът*

$$L_{halt} \;=\; \{\, \ulcorner \mathcal{M} \urcorner \sharp \omega \mid \textit{машината } \mathcal{M} \textit{ спира над } \omega \,\}$$

не е разрешим.

Доказателство. Допускаме, че L_{halt} е разрешим и нека $\mathcal{M}_{halt}$ е разрешител за L_{halt}. Ще построим разрешител $\mathcal{M}'$ за L_{accept}.

При даден вход ω:

(1) $\mathcal{M}'$ проверява дали $\omega \in L_{code}$ и ако не е вярно, $\mathcal{M}'$ отхвърля ω.

(2) Ако $\omega \in L_{code}$, $\mathcal{M}'$ изпълнява $\mathcal{M}_{halt}$ върху вход $\omega\sharp\omega$.

(3) Ако изпълнението завърши в q_{reject}, $\mathcal{M}'$ отхвърля ω.

(4) Ако изпълнението завърши в q_{accept}, $\mathcal{M}'$ симулира $\mathcal{M}_\omega$ над вход ω.

(5) Ако последната симулация завърши успешно, $\mathcal{M}'$ приема ω, ако завърши неуспешно, $\mathcal{M}'$ отхвърля ω.

Получихме, че $\mathcal{L}(\mathcal{M}') = L_{accept}$ и $\mathcal{M}'$ е разрешител, което е противоречие с факта, че L_{accept} е неразрешим език.

Идеята на доказателството е, че сведохме L_{accept} към L_{halt}. Успяхме да направим симулация на $\mathcal{M}_\omega$ над вход ω, която винаги завършва, тъй като чрез $\mathcal{M}_{halt}$ премахнахме случая, в който тази симулация зацикля.

Теорема 4 *Универсалният език*

$$L_{univ} \;=\; \{\, \ulcorner \mathcal{M} \urcorner \sharp \omega \mid \mathcal{M} \text{ е машина на Тюринг и } \omega \in \mathcal{L}(\mathcal{M}) \,\}$$

е полуразрешим.

Доказателство. Строим многолентова машина $\mathcal{U}$, която при вход α:

(1) Проверява дали α има вида $\ulcorner \mathcal{M} \urcorner \sharp \omega$ и ако не, $\mathcal{U}$ отхвърля α.

(2) Ако $\alpha = \ulcorner \mathcal{M} \urcorner \sharp \omega$, то $\mathcal{U}$ симулира работата на $\mathcal{M}$ върху вход ω.

Симулацията ще се извършва на отделна лента, на която първоначално копираме ω. Ще има още една допълнителна лента, върху която се пази думата 0^i, където q_i е текущото състояние на $\mathcal{M}$ (в началото на симулацията нейното съдържание е 0, тъй като $q_{start} = q_1$). На всяка стъпка $\mathcal{U}$ претърсва кода на $\mathcal{M}$ за необходимия преход, който започва с текущото състояние на $\mathcal{M}$ и кодът на текущата буква върху лентата за симулация. Като го намери, записва новото състояние на $\mathcal{M}$ и променя лентата за симулация по съответен начин.

(3) Тъй като $\mathcal{U}$ напълно копира поведението на $\mathcal{M}$:

$\mathcal{U}$ приема α точно когато $\mathcal{M}$ приема ω,

$\mathcal{U}$ отхвърля α точно когато $\mathcal{M}$ отхвърля ω,

$\mathcal{U}$ зацикля над α точно когато $\mathcal{M}$ зацикля над ω.

По този начин, $\mathcal{L}(\mathcal{U}) = L_{univ}$. Естествено е да наречем $\mathcal{U}$ *универсална машина на Тюринг*, тъй като това е една единствена машина на Тюринг, която може да симулира изчислението на всички машини на Тюринг. Счита се, че това е формалният първообраз на съвременните компютри и по-точно на идеята, че съдържанието на програмите трябва да се съхранява като вид данни в паметта.

Дефиниция 5 *Нека $K, L \subseteq \Sigma^\star$ и $f : \Sigma^\star \to \Sigma^\star$. Казваме, че функцията f свежда K към L, ако:*

$$\forall \omega \in \Sigma^\star \;:\; \omega \in K \iff f(\omega) \in L.$$

Ще отбелязваме, че $K \leq_m L$, ако съществува изчислима f, която свежда K към L.

Лесно се съобразяват следните две свойства:

1. $K \leq_m L$ и L е разрешим влече, че K е разрешим.
2. $K \leq_m L$ и L е полуразрешим влече, че K е полуразрешим.

Например, $L_{accept} \leq_m L_{univ}$ посредством $f(\omega) = \omega\sharp\omega$. Тъй като L_{accept} не е разрешим, L_{univ} също не е разрешим.

Разбира се, съществува по-общо свеждане на един език K към друг език L, при което в рамките на едно изчисление може да се отправят произволен брой заявки от вида $\alpha \in L$?, където α може да зависи от резултати от предходни заявки. Такова най-общо свеждане се нарича *Тюрингово свеждане*. При него L се третира, като зададен чрез *оракул*, който отговаря на заявките, въпреки че в общия случай няма алгоритъм за това.

Сложност

Машината на Тюринг $\mathcal{M}$ *работи за полиномиално време*, ако съществува полином $p(x) \in \mathbb{N}[x]$, такъв че

$$\forall \omega \in \Sigma^\star, \lambda, \rho \in \Gamma^\star,\ q \in Q\ :\ (\epsilon, q_{start}, \omega\sqcup)\ \vdash^{\ell}_{\mathcal{M}}\ (\lambda, q, \rho)\ \Rightarrow\ \ell \leq p(|\omega|).$$

С други думи, за произволен вход ω, дължината на всички клони в дървото на изчислението не може да надвишава $p(|\omega|)$. Очевидно е, че $\mathcal{M}$ е разрешител.

Означаваме:

$$\begin{aligned}\mathcal{P} \ =\ \{\ L \subseteq \Sigma^\star \mid L = \mathcal{L}(\mathcal{M}) &\text{ за някоя детерминирана } \mathcal{M},\\ &\text{която работи за полиномиално време }\},\\ \mathcal{NP} \ =\ \{\ L \subseteq \Sigma^\star \mid L = \mathcal{L}(\mathcal{N}) &\text{ за някоя недетерминирана } \mathcal{N},\\ &\text{която работи за полиномиално време }\}.\end{aligned}$$

Машината на Тюринг $\mathcal{M}$ *работи за експоненциално време*, ако съществува полином $p(x) \in \mathbb{N}[x]$, такъв че

$$\forall \omega \in \Sigma^\star, \lambda, \rho \in \Gamma^\star,\ q \in Q\ :\ (\epsilon, q_{start}, \omega\sqcup)\ \vdash^{\ell}_{\mathcal{M}}\ (\lambda, q, \rho)\ \Rightarrow\ \ell \leq 2^{p(|\omega|)}.$$

Нека

$$\begin{aligned}\mathcal{EXP} \ =\ \{\ L \subseteq \Sigma^\star \mid L = \mathcal{L}(\mathcal{M}) &\text{ за някоя детерминирана } \mathcal{M},\\ &\text{която работи за експоненциално време }\}.\end{aligned}$$

Знаем за следните включвания:

$$\text{крайни} \subseteq \text{регулярни} \subseteq \text{безконтекстни} \subseteq \mathcal{P} \subseteq \mathcal{NP} \subseteq \mathcal{EXP}.$$

Първите две включвания са строги. Третото включване се основава на факта, че комбинацията от построяването на нормална форма на Чомски с

CYK-алгоритъма за принадлежност може да се симулира за полиномиално време. Това включване е строго, тъй като езикът $\{a^n b^n c^n \mid n \in \mathbb{N}\} \in \mathcal{P}$ (за него вече построихме подходяща машина). Включването $\mathcal{NP} \subseteq \mathcal{EXP}$ следва от теоремата за симулация на недетерминирана машина с детерминирана. Забележете, че клоните на изчислението имат полиномиална дължина, но броят на тези клони е експоненциален и в симулацията всичките трябва да бъдат обходени (в най-лошия случай).

За последните две включвания въпросът дали са строги е отворен. Безспорно $\mathcal{P} = \mathcal{NP}$? е най-популярният отворен проблем в теоретичната информатика.

Теорема 6 *Езикът L принадлежи на $\mathcal{NP}$ тогава и само тогава, когато съществуват език $V \in \mathcal{P}$ и полином $q(x) \in \mathbb{N}[x]$, такива че*

$$\forall \omega \in \Sigma^\star \ : \ \omega \in L \iff \exists \alpha, \ |\alpha| \leq q(|\omega|) \ \ \& \ \ \omega \sharp \alpha \in V.$$

Доказателство. Естествено е да наречем V *валидатор* и α да е *свидетел* за ω. По този начин, една дума $\omega \in L$ точно когато за нея съществува свидетел α с подходяща дължина, който може да се валидира с V.

За посоката ($\Longrightarrow$). Нека $\mathcal{N}$ е недетерминирана машина, която работи за полиномиално време $p(x)$ и $\mathcal{L}(\mathcal{N}) = L$. Избираме $q(x) = p(x)$. За да опишем валидатора V използваме машината $\mathcal{D}$ от теоремата за детерминизация. Строи следната машина $\mathcal{M}$, при вход $\omega \sharp \alpha$:

(1) $\mathcal{M}$ копира ω върху лентата за симулация на $\mathcal{D}$;

(2) $\mathcal{M}$ копира α върху управляващата лента на $\mathcal{D}$;

(3) $\mathcal{M}$ извършва една итерация от цикъла на изчислението на $\mathcal{D}$, т.е. симулира машината $\mathcal{N}$ върху вход ω съобразно съдържанието α на управляващата лента.

(4) Ако симулацията завърши в q_{accept}, то $\mathcal{M}$ приема $\omega \sharp \alpha$.

(5) В противен случай, $\mathcal{M}$ отхвърля $\omega \sharp \alpha$ (напомням, това означава, че симулацията завършва в друго състояние или се стига до бленк или до символ, за който няма съответен преход върху управляващата лента).

Нека $V = \mathcal{L}(\mathcal{M})$. Ясно е, че $V \in \mathcal{P}$ (дори с полином от първа степен).

Нека $\omega \in L$. Тогава $(\epsilon, q_{start}, \omega \sqcup) \ \vdash_{\mathcal{N}}^{\leq q(|\omega|)} \ (\lambda, q_{accept}, \rho)$. Нека α кодира това изчисление (съобразно доказателството от теоремата за детерминизация на машина на Тюринг). Тогава $|\alpha| \leq q(|\omega|)$ и от построението на $\mathcal{M}$ е ясно, че $\omega \sharp \alpha \in V$. Обратно, ако $\omega \sharp \alpha \in V$, то $\omega \in L = \mathcal{L}(\mathcal{N})$, тъй като това е единственият случай, при който $\mathcal{M}$ завършва успешно.

За посоката ($\Longleftarrow$). Да изберем полином $q(x) \in \mathbb{N}[x]$ и език V с описаните свойства, $V = \mathcal{L}(\mathcal{M})$ и $\mathcal{M}$ работи за полиномиално време $p(x)$.

Строим следната недетерминистична машина $\mathcal{N}$, при вход $\omega \in \Sigma^\star$:

(1) Изчислява $1^{q(|\omega|)}$.

(2) Избира недетерминистично α с дължина $|\alpha| \leq q(|\omega|)$ (по-точно има преходи, гарантиращи че от подходяща конфигурация $(\lambda, q, \sqcup\rho)$ може за една стъпка да се стигне до $(\lambda, q, x\rho)$ за всяка буква x).

(3) Симулира $\mathcal{M}$ върху вход $\omega\sharp\alpha$.

Забележете, че: първата част отнема $q'(|\omega|)$ на брой стъпки, където q' е подходящ полином; втората част отнема не повече от $q(|\omega|)$ на брой стъпки; третата част отнема най-много $p(|\omega\sharp\alpha|) \leq p(|\omega| + 1 + q(|\omega|))$ стъпки. Така всеки клон на изчислението има дължина $\leq q'(|\omega|)+q(|\omega|)+p(|\omega|+1+q(|\omega|))$, което е полином на $|\omega|$. Получихме, че $\mathcal{N}$ работи за полиномиално време и също е ясно, че $L(\mathcal{N}) = L$, тъй като $\omega \in L(\mathcal{N})$ точно когато в изчислението има поне един успешен клон, което е изпълнено точно когато има свидетел с описаните свойства, т.е. когато $\omega \in L$.

Дефиниция 7 *За езици $K, L \in \Sigma^\star$, бележим $K \leq_m^P L$ ако $K \leq_m L$ и свеждащата функция f е изчислима с детерминистична машина на Тюринг за полиномиално време.*

Лесно се съобразява, че:
1. Ако $L \in \mathcal{P}$ и $K \leq_m^P L$, то $K \in \mathcal{P}$.
2. Ако $L \in \mathcal{NP}$ и $K \leq_m^P L$, то $K \in \mathcal{NP}$.

Дефиниция 8 *Един език $L \in \mathcal{NP}$ е $\mathcal{NP}$-пълен, ако за всеки $K \in \mathcal{NP}$ имаме $K \leq_m^P L$.*

Нека SAT е езикът, който се състои от булевите функции f в конюнктивна нормална форма, които приемат поне една стойност истина. Забележете, че $SAT \in \mathcal{NP}$, тъй като ролята на свидетел за f може да бъде съответната оценка на променливите, при която f приема стойност истина.

Теорема 9 (Кук-Левин) *Езикът SAT е $\mathcal{NP}$-пълен.*

Доказателството на теоремата симулира работата на машина на Тюринг с помощта на булева функция в конюнктивна нормална форма.

Други $\mathcal{NP}$-пълни езици са: (1) графите, в които има Хамилтонов цикъл; (2) редиците от естествени числа $a_1, \ldots, a_n, W$, такива че W е сума на някои от a-тата. Съществуват още стотици примери.

Можем да кажем, че $\mathcal{NP}$-пълните езици са най-сложните езици в класа $\mathcal{NP}$. Всеки два от тях се свеждат полиномиално един към друг. Важността им по отношение на проблема $\mathcal{P} = \mathcal{NP}$? произтича от следните еквивалентности:

$$\mathcal{P} = \mathcal{NP} \iff \text{за някой } \mathcal{NP}\text{-пълен език } L,\ L \in \mathcal{P},$$

$$\mathcal{P} \neq \mathcal{NP} \iff \text{за някой } \mathcal{NP}\text{-пълен език } L,\ L \notin \mathcal{P}.$$

Материалът за четиринадесетата лекция от записките:

глава 4.4 (**Кодиране на машина на Тюринг. Диагоналният език. Универсална машина на Тюринг**), стр. 183-186,

глава 4.9 (**Сложност**), стр. 206.

За допълнително четене от учебника на Hopcroft, Motwani, Ullman:

глава 9.1 (A Language That Is Not Recursively Enumerable), глава 9.2 (An Undecidable Problem That is RE), глава 10.1 (The Classes $\mathcal{P}$ and $\mathcal{NP}$), глава 10.2 (An $\mathcal{NP}$-Complete Problem).